

1. Pipeline stuck 2 days at an <code>input</code> step, blocking an agent — fix? Move <code>input</code> to a stage with <code>agent none</code> (doesn't hold executor); wrap in <code>timeout{}</code> to auto-abort; manually abort & re-trigger stuck build.	16. Deploy stage occasionally deploys stale image tag — root cause approach? Check if <code>docker build</code> cache reused an old layer without cache-busting on source change, or GitOps repo update raced with another build — enforce commit-SHA tagging + <code>disableConcurrentBuilds()</code> .
2. Design a pipeline for microservices repo, each service own Dockerfile/versioning. Multibranch/parameterized pipeline + shared library for common logic; <code>when { changeset }</code> path-based detection to build only affected services; matrix for multi-service parallel build.	17. How would you implement a rollback stage automatically on failed smoke test? <code>post { failure { sh 'helm rollback app' } }</code> at the deploy stage, or GitOps revert commit + push, triggered from <code>post</code> block.
3. SonarQube quality gate keeps timing out — cause? Webhook from SonarQube not reaching Jenkins (network/firewall), wrong webhook URL, or <code>waitForQualityGate</code> timeout too short for large analysis.	18. Migrate 200 Freestyle jobs to Pipeline-as-code — approach? Phase migration by team/criticality, extract common logic into shared library first, use Job DSL / Jenkins Job Builder to bulk-generate Multibranch jobs, validate parity before decommissioning Freestyle.
4. Roll back a failed K8s deployment triggered from Jenkins. GitOps: revert commit in GitOps repo (ArgoCD auto-syncs). Direct deploy: <code>helm rollback</code> or <code>kubectl rollout undo</code> in dedicated rollback stage.	19. A K8s agent pod fails to start — where do you look? Kubernetes plugin cloud config (namespace/RBAC), pod template resource requests vs cluster capacity, image pull errors/registry auth, Jenkins controller-to-K8s API connectivity.
5. Scale Jenkins for 50+ teams, dozens of builds/day. K8s dynamic agents (scale-to-zero), controller with 0 executors, shared libraries for standardization, folder-based RBAC, scoped credentials.	20. How to enforce that prod deploys only happen from the main branch? <code>when { branch 'main'; environment name: 'ENV', value: 'prod' }</code> guard on the deploy stage; reject params combination otherwise.
6. Prevent leaked secret from a compromised Jenkinsfile PR. Folder-scoped credentials (not global), branch protection + mandatory review on Jenkinsfile changes, <code>withCredentials</code> masking, restrict agent trust for fork PR builds.	21. Reduce pipeline runtime from 25 min to under 10. <code>parallel{}</code> independent stages (lint/unit/security scan), cache dependencies (Docker layer cache, Maven .m2), use K8s ephemeral agents sized correctly, skip unaffected microservices via <code>changeset</code> detection.
7. Blue-green vs canary — implement via Jenkins + Helm/K8s? Blue-green: deploy new version parallel service, swap ingress/service selector in one stage. Canary: gradual traffic % shift via Istio/Argo Rollouts with automated health-gated stage increments.	22. How do you handle a plugin upgrade breaking pipelines org-wide? Test upgrades in a staging Jenkins instance first, pin plugin versions, maintain plugin compatibility matrix, roll back via <code>JENKINS_HOME</code> plugin dir restore if needed.
8. Build fails intermittently on same agent, passes on retrigger — approach? Check for shared workspace/state contamination (concurrent builds), flaky external dependency (add <code>retry()</code>), resource exhaustion (disk/mem) on that agent, or non-idempotent test.	23. Pipeline needs to notify differently for dev vs prod failures. <code>post { failure { script { if(env.ENV=='prod'){ pagerduty() } else { slackSend() } } } }</code> — severity-based routing in <code>post</code> .
9. How to avoid duplicate deployments when 2 PRs merge close together? <code>disableConcurrentBuilds()</code> , queue-based serialization on the deploy stage, or lock resource via <code>LockableResources</code> plugin for the target environment.	24. How do you verify an image wasn't tampered with before prod deploy? Cosign signature verification as a K8s admission-controller policy (Kyverno/Sigstore policy-controller) — rejects unsigned/tampered images at deploy time, not just in CI.
10. Trivy scan blocks build on a CVE with no available fix yet — handle? Maintain a <code>.trivyignore</code> with justified CVE exceptions + expiry review date; don't blanket-ignore, track via risk acceptance ticket.	25. Onboarding a new team to Jenkins — what guardrails do you set day one? Folder with scoped RBAC + credentials, mandatory shared-library base pipeline, quota/timeout defaults via <code>options{}</code> , required Trivy + quality gate stages before any registry push.
11. Jenkins controller disk full from build artifacts — prevent recurrence. <code>buildDiscarder(logRotator())</code> retention policy, push artifacts to Nexus/Artifactory/S3 instead of archiving locally, scheduled <code>cleanWs()</code> , monitor disk via alerting.	
12. How do you test a Jenkinsfile change before merging to main? Use a feature branch in a Multibranch pipeline (auto-builds it independently), or Jenkins' "Replay" feature to test Groovy changes without committing.	
13. Secure secrets for a Terraform stage needing cloud credentials. <code>withCredentials</code> injecting short-lived STS tokens (assume-role) rather than static keys; never <code>terraform output</code> secrets to console; mask via credential binding.	
14. Pipeline needs different approval logic per environment (dev auto, prod manual). <code>when { expression { params.ENV == 'prod' } }</code> wrapping an <code>input</code> stage — only prompts for prod; dev/qa skip straight through.	
15. How to keep Jenkinsfile DRY across 30 microservice repos? Shared Library exposing a single <code>standardPipeline()</code> call each repo's Jenkinsfile invokes with repo-specific params — one place to fix/update CI logic org-wide.	

Real-Time Production-Grade Jenkinsfile Example

```
// Production CI/CD pipeline: build → scan → sign → push → GitOps deploy
pipeline {
    agent none

    options {
        timeout(time: 45, unit: 'MINUTES')
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '15'))
    }

    parameters {
        choice(name: 'ENV', choices: ['dev','qa','prod'], description: 'Target environment')
    }

    environment {
        IMAGE_NAME      = 'myorg/payments-service'
        ECR_REGISTRY     = '123456789.dkr.ecr.us-east-1.amazonaws.com'
        IMAGE_TAG        = "${env.GIT_COMMIT.take(7)}-${env.BUILD_NUMBER}"
        SONAR_TOKEN       = credentials('sonar-token-id')
    }

    stages {

        stage('Checkout') {
            agent { label 'build' }
            steps { checkout scm }
        }

        stage('Build & Unit Test') {
            agent { label 'build' }
            steps {
                sh 'mvn clean package'
            }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
        }

        stage('SonarQube + Quality Gate') {
            agent { label 'build' }
            steps {
                withSonarQubeEnv('SonarQubeServer') {
                    sh 'mvn sonar:sonar -Dsonar.projectKey=payments-service'
                }
                timeout(time: 5, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }

        stage('Docker Build') {
            agent { label 'docker' }
            steps {
                sh "docker build -t $IMAGE_NAME:$IMAGE_TAG ."
            }
        }

        stage('Trivy Scan') {
            agent { label 'docker' }
            steps {
                sh "trivy image --severity HIGH,CRITICAL --exit-code 1 $IMAGE_NAME:$IMAGE_TAG"
            }
        }

        stage('Push to ECR') {
            agent { label 'docker' }
            steps {
                withCredentials([[$class: 'AmazonWebServicesCredentialsBinding', credentialsId: 'aws-ecr-creds']]) {
                    sh '''
                        aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin $ECR_REGISTRY
                    '''
                }
            }
        }
    }
}
```

```
        docker tag $IMAGE_NAME:$IMAGE_TAG $SECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG
        docker push $SECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG
    ...
    }
}

stage('Sign Image (Cosign)') {
    agent { label 'docker' }
    steps {
        withCredentials([file(credentialsId: 'cosign-key', variables: 'COSIGN_KEY')]) {
            sh 'cosign sign --key $COSIGN_KEY $SECR_REGISTRY/$IMAGE_NAME:$IMAGE_TAG'
        }
    }
}

stage('Approve Production Deploy') {
    when { expression { params.ENV == 'prod' } }
    steps {
        input message: 'Deploy to production?', ok: 'Deploy', submitter: 'release-managers'
    }
}

stage('Helm Validate') {
    agent { label 'build' }
    steps {
        sh "helm lint ./charts/payments-service"
        sh "helm template ./charts/payments-service --values values-$(params.ENV).yaml > rendered.yaml"
        sh 'kubernetes validate --filename rendered.yaml'
    }
}

stage('Update GitOps Repo') {
    agent { label 'build' }
    steps {
        withCredentials([usernamePassword(credentialsId: 'gitops-repo-creds', usernameVariable: 'GIT_USER', passwordVariable: 'GIT_PASS')]) {
            sh '''
                git clone https://$GIT_USER:$GIT_PASS@github.com/myorg/gitops-repo.git
                cd gitops-repo
                yq -i '.image.tag = "$IMAGE_TAG"' apps/payments-service/values-$(ENV).yaml
                git commit -am "Deploy payments-service $IMAGE_TAG to $(ENV)"
                git push
            '''
        }
    }
}

post {
    always {
        node('build') { cleanWs() }
    }
    success {
        slackSend channel: '#ci-cd', color: 'good',
            message: "✓ ${env.JOB_NAME} #${env.BUILD_NUMBER} deployed $IMAGE_TAG to ${params.ENV}"
    }
    failure {
        slackSend channel: '#ci-cd', color: 'danger',
            message: "✗ ${env.JOB_NAME} #${env.BUILD_NUMBER} FAILED - check ${env.BUILD_URL}"
    }
}
}
```